3. ВКАЗІВКИ ДО ВИКОНАННЯ ЗАВДАННЯ

3.1. Розробка моделі та її експериментального дослідження

3.1.1. Розробки концептуального опису моделі

Концептуальна модель системи – це деталізований опис складових елементів системи, правил їх взаємодії та процесу, який моделюється, разом із зазначенням мети моделювання. Обов'язково на етапі розробки концептуальної моделі системи визначають набори вхідних змінних моделі, її параметрів та її вихідних характеристик. Усі числові значення, які використовують в описі моделі, мають бути приведені до співвідносних одиниць вимірювання.

Для кожної вхідної змінної моделі мають бути вказані:

- початкове значення;
- допустимий діапазон варіювання, тобто множина значень (неперервна або дискретна), в межах якої може змінюватись вхідна змінна в реальних умовах функціонування системи;
- закон розподілу та числові значення закону розподілу, якщо змінна приймає стохастичні значення.

Параметрами є числові характеристики процесу, які вважаються такими, що не можуть бути змінені, але їх значення має бути враховано в алгоритмі імітації. Наприклад, кількість етапів технологічної обробки деталі, кількість смуг руху на транспортній ділянці, обмеження швидкості руху на дорозі, продуктивність каналу передачі даних і т.і. Для кожного параметра моделі має бути вказано числове значення у відповідних одиницях вимірювання.

Для кожної вихідної змінної має бути вказаний її зміст і спостереження яких числових значень в ході імітації вона потребує. Наприклад, для визначення продуктивності виробничої ділянки необхідно знати кількість виробів, вироблених в одиницю часу.

Оскільки імітується стохастичний процес у більшості завдань курсової роботи, то точність визначення вихідного значення досягається за рахунок повторних замірів вихідного значення.

Для опису процесу, який моделюється, зручно використовувати схематичне зображення послідовності подій, що відбувається в системі за умов, визначених текстом завдання. Зокрема, схематичне зображення процесу, що відтворюється, можна представити у вигляді діаграми BPMN (Business Process Model and Notation) [2].

Якщо за текстом завдання потрібно розглянути альтернативні моделі, то для всіх моделей мають бути розроблені концептуальні описи, які в подальшому розвивають у відповідні формалізовані описи та алгоритми імітації.

3.1.2. Розробка формалізованого опису моделі

Формалізована модель системи – це опис елементів системи та процесу, який моделюється, в термінах формалізму, який обраний при виборі завдання. Для мережі масового обслуговування елементами формального опису є пристрої обслуговування, черги та маршрути слідування. Для стохастичної мережі Петрі – позиції, переходи та дуги. Для Петрі-об'єктної моделі – множина Петрі-об'єктів з визначеними зв'язками між ними.

Для всіх елементів формального опису моделі мають бути визначені усі їх числові параметри. Наприклад, для пристрою мережі масового обслуговування має бути визначений час обробки, а для переходу стохастичної мережі Петрі – часова затримка в переході та параметри, які важливі для вирішення конфлікту переходів – пріоритет та ймовірність запуску.

Для всіх вхідних змінних та параметрів моделі з переліку, визначеного у концептуальній моделі, обов'язково мають бути визначені числові значення, а для випадкових величин – формули генераторів випадкових чисел.

Для всіх вихідних характеристик моделі з переліку, визначеного у концептуальній моделі, обов'язково мають бути визначені формула обчислення за даними, які обчислюється в процесі імітації, та вимоги до точності визначення вихідного значення.

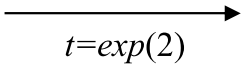
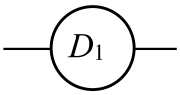
Елементи формального опису мережею масового обслуговування та визначені для них числові параметри представлені у таблиці 3.1. Мережа масового обслуговування складається з систем масового обслуговування (СМО), з'єднаних дугами, що представляють маршрут слідування запитів у відповідності до технологічного процесу обробки, та зовнішнього потоку надходження запитів. Кожна СМО – це один або кілька ідентичних пристроїв обслуговування з чергою для очікування або без неї. За відсутності черги перед пристроєм або за відсутності вільного місця в обмеженій черзі запит отримує відмову в обслуговуванні. Зовнішнє надходження присутнє у розімкнутих мережах масового обслуговування, але у замкнутих мережах цей елемент відсутній. Замість обробки зовнішнього потоку запитів у розімкнутій мережі, у замкнутій відбувається циркуляція запитів в мережі так, що в процесі обробки запити переходять від однієї СМО до іншої.

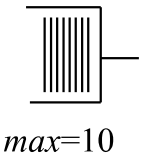
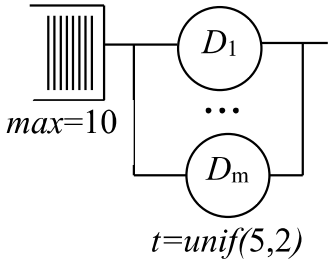
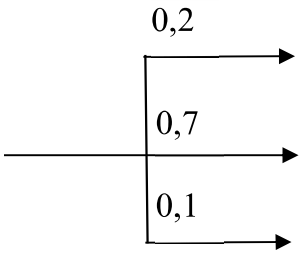
Окрім класичних елементів (черга, пристрій, дуга) у таблиці 3.1 до формального опису мережі масового обслуговування додані такі елементи: «блокування маршруту» для управління маршрутом слідування до СМО, «дуга з числовим параметром» для групування запитів при вході на обробку у СМО або виході з неї. Блокування маршруту супроводжується формулюванням умови блокування, яка формулюється у вигляді предикату або логічного виразу. Отже, при досягненні значення 1 для умови відбувається блокування маршруту, в іншому випадку маршрут відкритий для слідування. За рахунок формулювання умови блокування маршруту надається можливість гнучко налаштовувати можливі маршрути між СМО. Дуга з числовим параметром використовується для групування кількох елементів в один та розгрупування одного елемента у кілька ідентичних. Якщо на дузі між СМО1 та СМО 2 вказаний параметр 5, то 5 запитів, які виходять зі СМО1, перетворюються на дузі в 1 запит (групуються), що надходить на обробку у СМО2. Якщо на дузі вказаний параметр 1/5, то 1 запит, який вийшов зі СМО1 перетворюється у 5 запитів (розгруповується), що спрямовуються на обробку у СМО2.

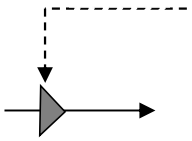
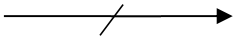
Домовимось про такі позначення величин для опису числових параметрів:

- d – детерміноване значення,
- $exp(d)$ – випадкова величина, розподілена за експоненціальним законом з середнім значенням d,
- $norm(d, s)$ – випадкова величина, розподілена за нормальним законом з середнім значенням d та середнім квадратичним відхиленням s,
- $unif(d, s)$ – випадкова величина, рівномірно розподілена в інтервалі $(d-s, d+s)$,
- $lnorm(d, s)$ – випадкова величина, розподілена за логнормальним законом з середнім значенням d та середнім квадратичним відхиленням s,
- $erl(d, s)$ – випадкова величина, розподілена за законом Ерланга з середнім значенням d та середнім квадратичним відхиленням s.

Таблиця 3.1. Елементи формального опису мережі масового обслуговування.

Елемент	*Графічне позначення*	*Зміст*	*Числові параметри*
Надходження запитів ззовні		Надходження запитів на обслуговування з заданою часовою затримкою. Якщо надходження запитів ззовні немає, то цей елемент може бути відсутнім.	t - часова затримка в умовних одиницях часу, між послідовними надходженнями запитів на обслуговування, яка може бути вказана детермінованим значенням або випадковою величиною з заданим законом розподілу. Для мережі без зовнішнього надходження має бути вказана кількість запитів, що циркулюють в ній.
Пристрій обслуговування	 $t=unif(5, 2)$	Пристрій, названий D_1, обробляє запити по одному з часовою затримкою t	t - часова затримка в умовних одиницях часу, необхідна для обробки одного запиту, яка може бути вказана детермінованим значенням або випадковою величиною з заданим законом розподілу

Елемент	*Графічне позначення*	*Зміст*	*Числові параметри*
Черга		Накопичувач запитів, що очікують звільнення пристроя обслуговування.	*max* – обмеження на кількість місць в черзі, яке може бути задане цілим числом. За замовчуванням (тобто якщо параметр не вказаний), кількість місць в черзі не обмежена.
Система масового обслуговування (СМО)		Один або кілька ідентичних пристроїв з чергою перед ними. Черга є складовою частиною СМО, тому з’єднання між чергою та пристроями не дуга, а відрізок.	*m* - кількість пристроїв у СМО
Дуга		Маршрут слідування запиту до наступної СМО	За замовчуванням ймовірність слідування по маршруту 1.
Розгалуження маршруту		Маршрут слідування запиту до однієї з наступних СМО, вибір якої здійснюється за заданими ймовірностями.	Ймовірності вибору кожного маршруту слідування, які у сумі складають 1.

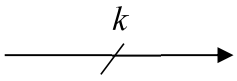
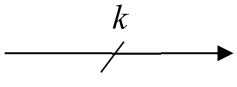
Елемент	*Графічне позначення*	*Зміст*	*Числові параметри*
Блокування маршруту	*B(S,t)* 	Встановлює блокування маршруту слідування у наступну СМО, якщо *B(S,t)*=1. Запит залишається у пристрої, з якого намагається вийти. Пристрій переходить у стан блокований (час обробки запиту завершився, але залишити пристрій немає можливості). Маршрут відкритий тільки за умови *B(S,t)* = 0.	$B(S,t)$ – предикат, який в найбільш загальному випадку залежить від стану S моделі та/або поточному моменту часу *t*. Наприклад, якщо умовою блокування маршруту є досягнення черги свого обмеження L_{max}, то предикат має вигляд $B(S,t) = \left(L_j(t) = L_{max}\right)$.
Дуга з числовим параметром	k 	Групування елементів у вказаній кількості	Якщо на дузі вказаний параметр k, то k запитів перетворюються на дузі в 1 запит (групуються), що спрямовується далі по маршруту. Якщо на дузі вказаний параметр 1/k, то 1 запит перетворюється у k запитів (розгруповується), що спрямовуються далі по маршруту.

Елементи формального опису стохастичною мережею Петрі (з багатоканальними та конфліктними переходами) та визначені для них числові параметри представлені у таблиці 3.2. Кожній події, що відбувається у системі,

ставиться у відповідність перехід. Умови для виконання подій відтворюються у вхідних позиціях переходу. Умови, які є результатом того, що подія відбулась, відтворюються у вихідних позиціях переходу. Просте правило спрацьовування переходу забезпечує надзвичайно високу гнучкість для відтворення різноманітних подій. За наявності в усіх вхідних позиціях переходу кількості маркерів у кількості не менше кратності дуги, що з'єднує позицію з переходом, умова спрацьовування переходу виконана. Коли перехід спрацьовує, то з усіх його вхідних позицій маркери віднімаються у кількості, рівній кратності дуги, що з'єднує позицію з переходом, та запам'ятовується момент виходу з переходу. Коли настає момент виходу з переходу, то він вилучається з переліку запам'ятованих, а в усі вихідні позиції переходу маркери додаються у кількості, рівній кратності дуги, що з'єднує перехід з позицією.

Зауважимо, що в кожному переході зберігається список моментів виходу з переходу. Це означає, що переходи мережі Петрі вважаються багатоканальними за замовчуванням. Якщо за текстом завдання потрібно обмежити кількість входів в перехід, то виконувати це потрібно за допомогою умови, що вказує на наявність вільних місць для входу в перехід.

Таблиця 3.2. Елементи формального опису стохастичною мережею Петрі.

Елемент	*Графічне позначення*	*Зміст*	*Числові параметри*
Позиція		Відтворює виконання умов для певних подій.	m – кількість маркерів, що вказує на виконання умови.
Вхідна дуга		З'єднує позицію та перехід. Слугує для визначення умов, які мають бути виконані для спрацьовування переходу.	k – кількість маркерів, що віднімаються з позиції при спрацьовуванні переходу.
Вихідна дуга		З'єднує перехід та позицію. Слугує для визначення умов, які є наслідком спрацьовування переходу.	k – кількість маркерів, що додаються до позиції при спрацьовуванні переходу.

Елемент	*Графічне позначення*	*Зміст*	*Числові параметри*
Перехід	 *d* *r, b*	Відтворює зміну стану моделі при виконанні події за рахунок зміни свого стану та стану позицій, які є його вхідними та вихідними позиціями. Використовує параметри пріоритету та ймовірності запуску для вирішення конфлікту з іншими переходами, що виникає, якщо для них в даний момент часу теж виконана умова спрацьовування.	*d* – часова затримка в умовних одиницях часу, яка може бути вказана невід'ємним детермінованим значенням або випадковою величиною з заданим законом розподілу, *r* – пріоритет переходу, який може бути заданий цілим числом (за замовчуванням 0), більше значення параметру відповідає більш високому пріоритету, *b* – ймовірність спрацьовуваня переходу (за замовчуванням 1).

У порівнянні з мережею масового обслуговування мережа Петрі має низку переваг:

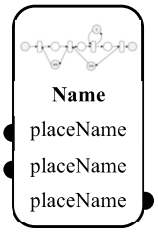
- можливість опису різноманітних умов для події завдяки різним вхідним позиціям;
- можливість моделювання часткового використання ресурсу завдяки захопленню заданої кількості маркерів з позиції, яка представляє наявну кількість ресурсу;
- можливість моделювання використання спільного ресурсу кількома процесами споживачами завдяки використанню однієї й тієї ж вхідної позиції, що представляє наявну кількість ресурсу, кількома переходами, що моделюють подію захоплення ресурсу процесом-споживачем;

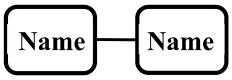
– можливість гнучкого налаштування вирішення конфліктів подій завдяки параметрам пріоритету та ймовірності запуску переходу.

Проте мережа Петрі часто призводить до великої кількості графічних елементів, які необхідні для представлення моделі, що ускладнює процес розробки, сприйняття моделі та її відлагодження. Цей недолік вирішує підхід Петрі-об’єктного моделювання.

Елементи формального опису Петрі-об’єктної моделі та визначені для них числові параметри представлені у таблиці 3.3. Графічне представлення Петрі-об’єктів та зв’язків між ними використовуємо за граматикою мови візуального програмування Петрі-об’єктної моделі, запропонованою в роботі [13].

Таблиця 3.3. Елементи формального опису Петрі-об’єктної моделі.

Елемент	*Графічне позначення*	*Зміст*	*Числові та інші параметри*
Стохастична мережа Петрі	Складається з елементів мережі Петрі, визначених у таблиці 3.2	Опис динаміки функціонування окремого об’єкта, що використовуєтсья для конструювання моделі	Параметри елементів мережі Петрі, які визначені у таблиці 3.2
Петрі-об’єкт		Петрі-об’єкт *Name*, який створений з заданою мережею Петрі. Петрі-об’єкт з вказівкою на позиції *placeName*, що призначені для ототожнення з іншими Петрі-об’єктами.	Усі числові параметри, що характеризують його функціональність, містяться у мережі Петрі об’єкта.
Ототожнювач позицій між Петрі-об’єктами		Ототожнення вказаних позицій *placeName* мереж Петрі-об’єктів	Немає

Елемент	*Графічне позначення*	*Зміст*	*Числові та інші параметри*
Зв'язок між Петрі-об'єктами		Зв'язок між Петрі-об'єктами *Name*, що склдається з одного або більше ототожнень позицій мереж Петрі-об'єктів	Немає
Група Петрі-об'єктів		Група Петрі-об'єктів	*n* – кількість Петрі-об'єктів в групі

Наведемо приклад опису усіма трьома формалізмами для такої системи:

Обробка деталей здійснюється на 5 ідентичних роботах, що розміщені уздовж стрічки конвеєра. Деталь, яка надходить, намагається потрапити на обробку першим роботом і, якщо він вільний, то обробляється і залишає систему. У противному випадку деталь намагається зайняти єдине вільне місце в очікуванні перед роботом, якщо і це не вдається, то деталь рухається далі по конвеєру до наступного робота і т.д. Якщо останній 5-ий робот виявився зайнятим, то виконується переміщення деталі на початок стрічки конвеєра. Метою моделювання є визначення продуктивності виробничої системи та середнього часу, що проводить деталь на виробничій ділянці, при таких заданих величинах: інтенсивність надходження деталей на обробку складає в середньому 4 деталі/хвилину, інтенсивність обробки деталі роботом складає в середньому 1 деталь/хвилину хвилини, інтервал часу, який необхідний для переїзду до наступного уздовж стрічки робота – 30 секунд, інтервал часу, який необхідний для переміщення деталі на початок конвеєра – 3 хвилини.

На рисунках 3.1-3.3 надано формальний опис цієї виробничої системи мережею масового обслуговування, стохастичною мережею Петрі та у вигляді Петрі-об'єктної моделі відповідно. Вихідною характеристикою моделі за умовою є продуктивність, яка для усіх варіантів формалізованого опису визначається за формулою:

$$Productivity = \frac{\sum_{i=1}^{5} N_i}{T_{mod}},$$

де N_i – кількість оброблених і-вим роботом деталей, T_{mod} – час моделювання.

Іншою вихідною характеристикою є середній час, що проводить деталь на виробничій ділянці. Для її розрахунку підсумуємо час, проведений деталями на

всіх ділянках свого технологічного маршруту, та поділимо його на кількість оброблених деталей:

$$ProcessingTime = \frac{\sum_{i=1}^{5}\sum_{i}(Q_{ij} + D_{ij} + S_{ij})\Delta t_j}{\sum_{i=1}^{5} N_i},$$

де Δt_j – j-ий інтервал часу імітації, Q_{ij} – стан черги перед i-им роботом в j-ому інтервалі часу, D_{ij} – стан i-ого роботу в j-ому інтервалі часу, S_{ij} – стан ділянки конвеєра на переїзді від (i-1)-ого до i-ого роботу в j-ому інтервалі часу.

При такому підрахунку може бути неточність через враховування часу, витраченого на деталі, які ще на момент завершення імітації не завершили свою обробку. Проте при достатньо великій кількості оброблених деталей цією неточністю можна знехтувати. За необхідності, розрахунок можна уточнити враховуванням доданків $(Q_{ij} + D_{ij} + S_{ij})\Delta t_j$ тільки для тих інтервалів часу, які відбулись до моменту завершення обробки останньої обробленої i-им роботом деталі.

Розглянемо детально кожний формалізований опис. За формалізації мережею масового обслуговування (рис. 3.1) кожній ділянці технологічного маршруту обробки деталі потрібно поставити у відповідність СМО. Обробку деталей роботом представлятимемо одноканальною СМО з чергою обмеженої довжини 1, а переїзд деталі від одного робота до іншого багатоканальною СМО без черги. Оскільки в умові не вказано обмеження на кількість деталі, що можуть перевозитись конвеєром одночасно, то використовується нескінченна кількість каналів обробки. Таким чином, будь-яка деталь у будь-який момент часу завжди знаходить вільне місце на конвеєрі для свого транспортування до наступного робота.

На рисунку 3.1 використані такі позначення: D_i – i-ий пристрій одноканальної СМО, що моделює обробку деталі роботом, $C_{j1}…C_{j\infty}$, j=(i+1)%5, i=1..5 – канали багатоканальної СМО, що моделюють перевезення деталі по конвеєру до наступного вздовж стрічки робота або переміщення деталі до першого робота, якщо досягнутий кінець стрічки.

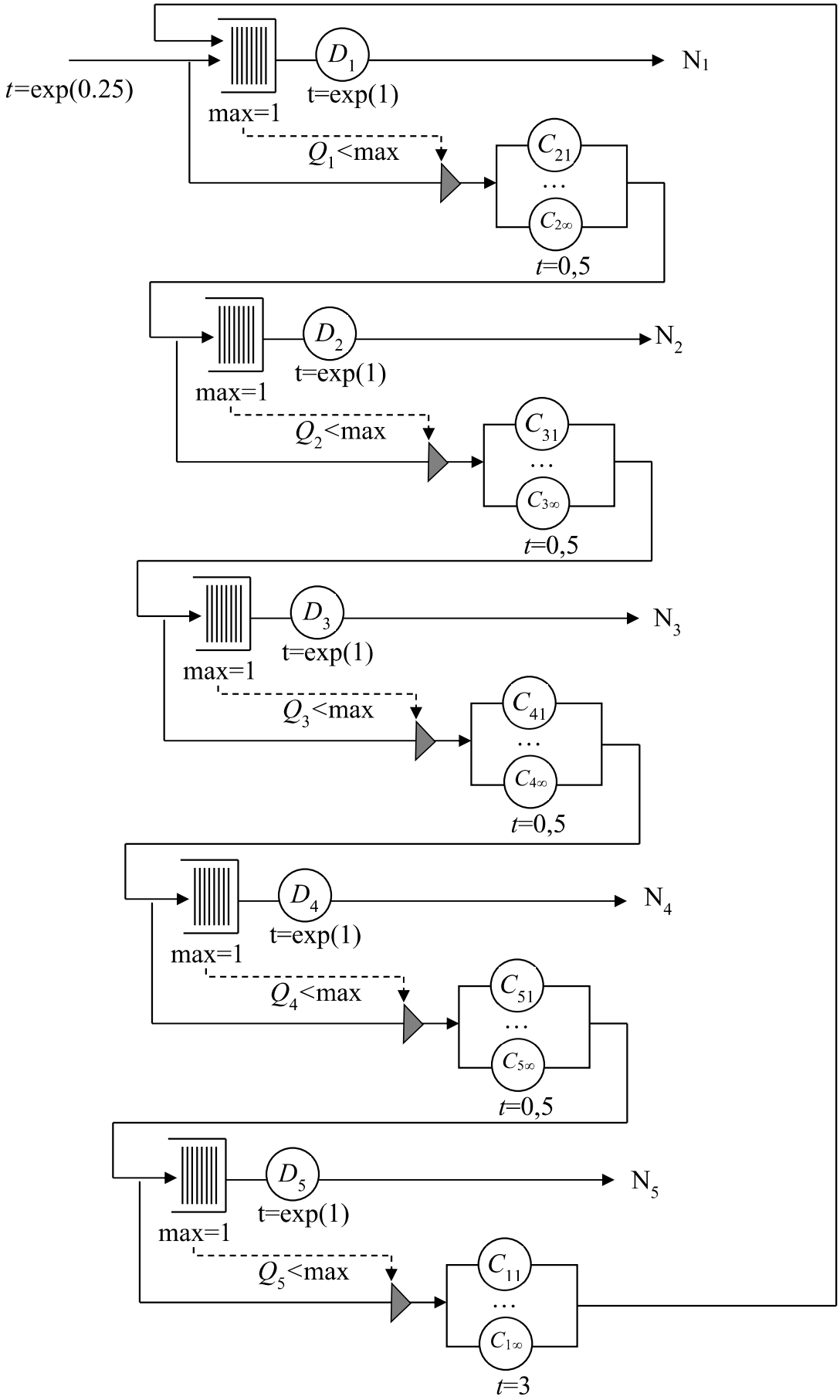


Рисунок 3.1. Мережа масового обслуговування, що формально представляє процес обробки деталей роботами, розміщеними уздовж стрічки конвеєра

За умовою задачі перед кожним роботом потрібно реалізувати розгалуження маршруту та визначити вибір маршруту в місцях розгалуження так, щоб продовжувати переїзд по конвеєру до наступного робота тільки у тому випадку, коли вичерпані місця в очікування біля певного робота. Саме для такого вибору за умовою передбачений елемент мережі масового обслуговування «блокування маршруту». Умова блокування $Q_i < max$ на шляху до наступного робота буде забороняти слідування по маршруту у разі, коли є місце для очікування в черзі перед *i*-им пристроєм (max – це встановлене умовою обмеження на кількість місць в черзі). В альтернативному випадку, маршрут буде відкритим.

Часові затримки для надходження та пристроїв вказані у відповідності до умови задачі. При виході з пристроя D_i підраховується кількість N_i успішно оброблених *i*-им пристроєм.

Зауважимо, що хоч за умовою задачі один і той самий конвеєр використовується для переїзду між роботами, використання однієї й тієї ж багатоканальної СМО для відтворення переїзду по конвеєру неможливе, оскільки в такому випадку на виході з цієї СМО неможливо визначити подальший маршрут слідування вимоги. Тому обраний варіант, в якому окрема багатоканальна СМО відтворює переїзд між окремими роботами.

Для розрахунку вихідних характеристик стан ділянки конвеєра в *j*-ий інтервал часу отримаємо за станом відповідної багатоканальної СМО:

$$S_{ij} = \sum_{k=1}^{\infty} C_{ikj}$$

де C_{ikj} – стан *k*-ого каналу багатоканальної СМО, що моделює перевезення деталі до *i*-ого роботу, в *j*-ому інтервалі часу.

В комп'ютерній реалізації моделі нескінченну кількість каналів потрібно реалізовувати кількістю каналів, достатньо великій для того, щоб ніколи не виникало відмови через зайнятість усіх каналів обслуговування.

За формалізації стохастичною мережею Петрі (рис. 3.2) кожній події ставимо у відповідність перехід мережі Петрі. Маємо такі події:

– "Надходження",
– "На обробку до *i*-ого роботу", *i*=1..5,
– "Обробка деталі *i*-им роботом", *i*=1..5,
– "Переїзд до *j*-ого роботу", *j*=(*i*+1)%5, *i*=1..5.

На рисунку 3.2 назви цих подій наведено скорочено, а деякі назви подій та переходів пропущено для спрощення сприйняття графічного представлення. Альтернатива переходів "На обробку до *i*-ого роботу", "Переїзд до *j*-ого роботу" вирішується наданням більш високого пріоритету переходу "Переїзд до *j*-ого роботу". Коли в позиції "Черга" буде 1 маркер, буде виконана умова для запуску одночасно для обох цих переходів, але оскільки для переходу "Переїзд до *j*-ого роботу" вказаний вищий пріоритет, то спрацьовує саме він. В іншому випадку буде спрацьовувати перехід "На обробку до *i*-ого роботу". Для кожного переходу визначена трійка параметрів: часова затримка, значення пріоритету та значення ймовірності запуску (таблиця 3.4). Числові значення вказані у форматі, в якому вони задаються у програмі (десяткова частина числа відокремлена крапкою замість коми).

Таблиця 3.4. Параметри переходів мережі Петрі, що моделює обробку деталей роботами.

Назва переходу	*Часова затримка*	*Значення пріоритету*	*Значення ймовірності запуску*
Надходження	exp(0.25)	0	1.0
На обробку до *i*-ого роботу	0.0	0	1.0
Обробка деталі *i*-им роботом	exp(1.0)	0	1.0
Переїзд до *j*-ого роботу	$\begin{cases} 0.5 \leftarrow j < 5 \\ 3.0 \leftarrow j = 1 \end{cases}$	1	1.0

Позиція "D_i", i=1..5 слугує для перевірки умови, що робот не зайнятий обробкою деталі. За наявності маркера в цій позиції та в позиції "Q_i" умова запуску для переходу "Обробка деталі *i*-им роботом" виконана. При запуску переходу, маркер буде вилучено з позиції " D_i" і, як наслідок, надходження нової деталі на обробку заборонено. Таким чином, ця умова забезпечує, що обробка деталей здійснюється роботом по одній.

Якщо порівняти з представленням мережею масового обслуговування, то представлення стало більш компактним за рахунок того, що кожний переїзд до наступного роботу відтворюється одним переходом мережі Петрі замість багатоканальної СМО з необмеженою кількістю каналів.

Для розрахунку вихідних характеристик моделі значення кількості оброблених деталей кожним *i*-им роботом отримаємо за кінцевим маркірування позиції "N_i", стан роботу та його черги в *j*-ому інтервалі визначимо за поточним станом позицій "D_i" та "Q_i", а стан ділянки конвеєра можемо визначити за станом переходу "Переїзд до *i*-ого роботу":

$$S_i = \left| E_{\text{Переїзд до і–ого}} \right| ,$$

де $|\cdot|$ позначено кардинальне число (або потужність) множини моментів виходів з переходу $E_{\text{Переїзд до і–ого}}$.

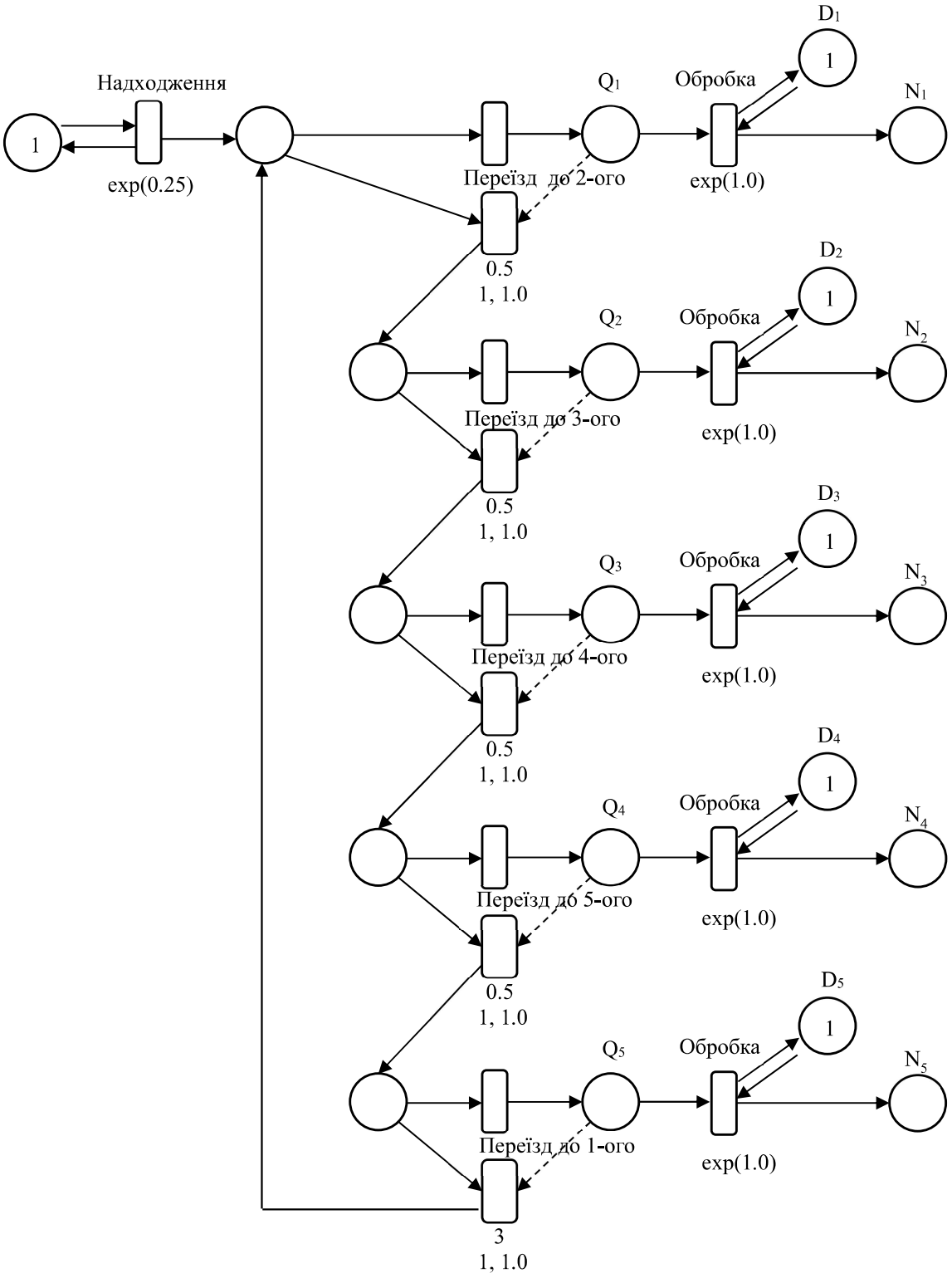


Рисунок 3.2. Стохастична мережа Петрі, що формально представляє процес обробки деталей роботами, розміщеними уздовж стрічки конвеєра

За формалізації Петрі-об’єктною моделлю (рис. 3.3) мають бути виділені типові фрагменти для відтворення процесу. Для задачі про обробку деталей роботами – це надходження деталей та обробка деталі. Назвемо відповідні Петрі-об’єкти "Generator" і "Robot" та розробимо їх мережі Петрі. Оскільки ці фрагменти схожі на ті, що використані за формалізації стохастичною мережею Петрі, напевно, вони не потребують додаткових роз’яснень. Розроблені мережі Петрі зберігаються з указуванням параметрів, які варіюються при тиражуванні об’єктів. Для об’єкту "Robot" таким параметром є часова затримка в переході "Переїзд до наступного", позначимо її r.

За наступним кроком створюються Петрі-об’єкти у потрібній кількості:

– "Generator", $i = 1$;

– "Robot", $i = 1…n$, $r = \begin{cases} 0.5 & \leftarrow i < n \\ 3.0 & \leftarrow i = n \end{cases}$.

Ототожнення позицій встановлюються для пар об’єктів Generator – Robot, Robot i - Robot $(i+1)$ % n, $i = 1…n$. В результаті з’єднання Петрі-об’єктів утворюється модель, яка представлена на останній діаграмі рисунку 3.3.

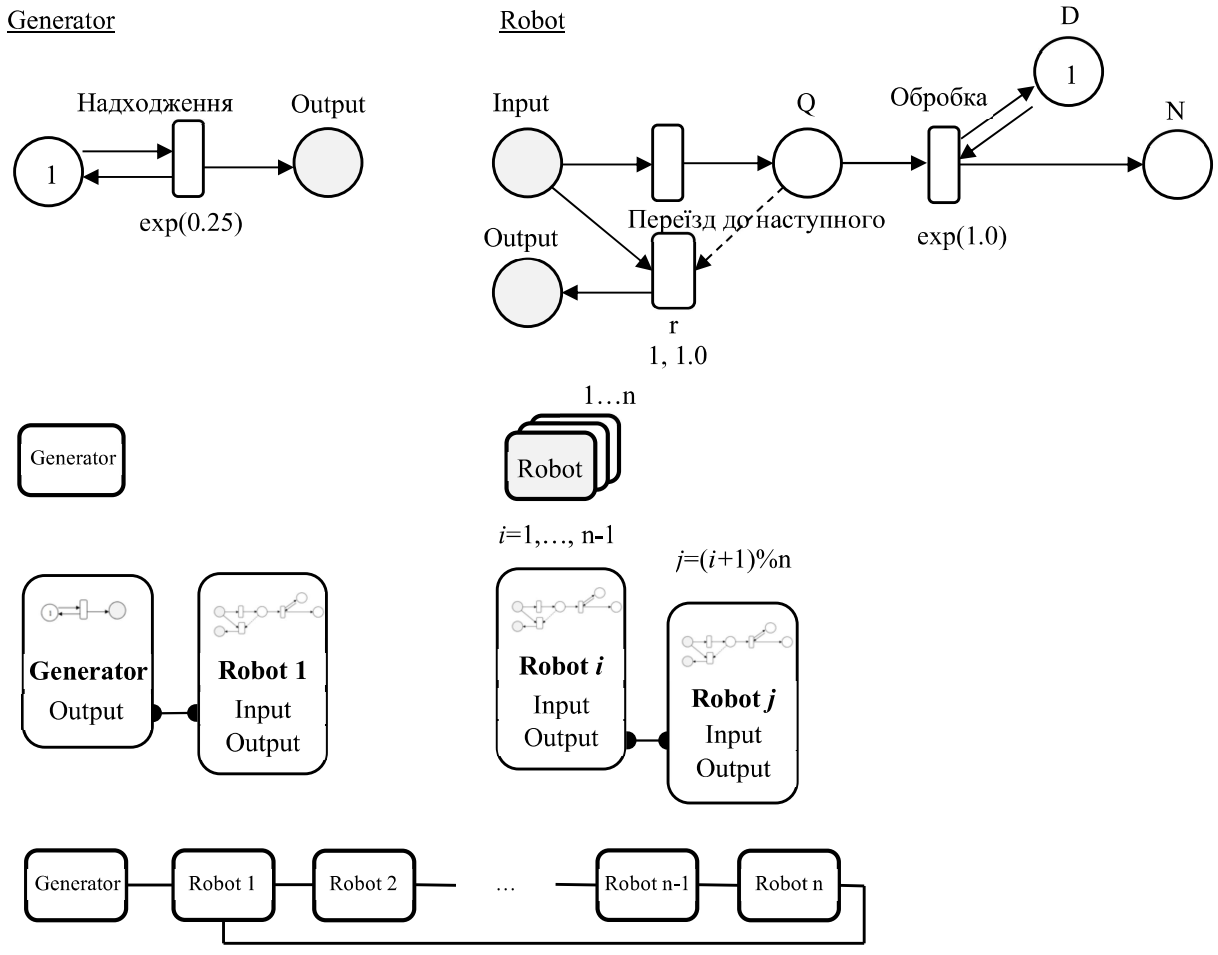


Рисунок 3.3. Формальний опис процесу обробки деталей роботами, розміщеними уздовж стрічки конвеєра, у вигляді Петрі-об’єктної моделі

У порівнянні зі звичайною стохастичною мережею Петрі формалізація у вигляді Петрі-об'єктної моделі дає змогу використовувати фрагменти мережі Петрі для створення нових з заданими параметрами у заданій кількості (тиражування об'єктів), скорочувати за рахунок цього кількість розроблюваних графічних елементів для побудови моделі, з чого випливає зменшення кількості помилок та зменшення витрат часу на розробку моделі. На додаток, представлення моделі стає більш компактним, що полегшує його сприйняття.

Отже, вибір формалізму суттєво впливає на представлення моделі. Формальне представлення моделі є основою для реалізації алгоритму імітації: набір елементів та правила їх функціонування мають відтворюватись алгоритмом імітації у відповідності до формального представлення.

3.1.3. Розробка алгоритму імітації

Розробка алгоритму імітації будь-якої системи потребує від розробника вирішення трьох проблем: просування часу, просування моделі в часі та збір статистичної інформації. Узагальнено алгоритм імітації складається з таких кроків:

- введення даних,
- доки поточний момент часу менший за час моделювання виконувати:
  - визначити найближчу подію,
  - зібрати статистичну інформацію на інтервалі між поточним моментом та наступним,
  - просунути час в момент найближчої події,
  - виконати зміну стану моделі, що відповідає найближчій події.
- виведення результатів моделювання.

У вигляді псевдокоду цей алгоритм має такий опис:

Algorithm 3.1 Discrete event system simulation

input input variables X, input parameters P, current time t=0,
modeling time Tmod
uses method of state transformation E(S,t)
initialize current state of model S
while t<Tmod
    find the moment of the nearest event t_{next}
    update statistics of output variables Y
    move the current time t:= t_{next}
    call state transformation E(S,t)

move the state of the model S:=E(S,t)
endwhile
calculate statistics of output variables Y
output variables Y

Алгоритм імітації має бути побудований за принципами, які є найбільш ефективними для побудови алгоритму імітації дискретно-подійної системи, а саме, з просуванням часу до найближчої події та з просуванням моделі в часі за подіями. Просування часу з дискретним кроком не може бути використано, оскільки у випадку стохастичної системи призводить або до великої кількості непродуктивних кроків або до неточного відтворення. Дійсно, крок має бути настільки маленьким, щоб в ньому відбувалось не більше однієї події. Але при стохастичних часових затримках можуть виникати як дуже маленькі інтервали між сусідніми подіями, так і дуже великі. Якщо для рівномірного розподілу існує найменш можливе значення, то для експоненціального розподілу немає найменш можливого значення. Намагання округлити значення до певного найменшого детермінованого кроку призводить взагалі до втрати точності відтворення подій. Отже, просування до найближчої події в алгоритмі імітації є найбільш ефективним як з погляду швидкодії, так і з погляду точності відтворення подій.

Зауважимо, що якщо можливою є ситуація, коли кілька подій відбуваються в однаковий момент часу, то варто подумати про необхідність враховувати пріоритет подій для правильного відтворення послідовності подій. Наприклад, якщо події «надходження нового запиту на обслуговування» і «завершення обслуговування в пристрої масового обслуговування» відтворюються саме у такому порядку (у випадку, коли випало, що вони мають відбутись в однаковий момент часу), то може статись ситуація, коли новий запит отримає відмову через обмеження на довжину чергу, а на наступному кроці алгоритму у цей же момент часу відбудеться звільнення пристрою. Якщо в алгоритмі завжди встановлений однаковий порядок виконання подій, а таких співпадінь моментів часу для виконання подій багато, то це може призвести до набагато більшої кількості відмов, ніж очікується і ніж спостерігається в реальній системі. Найбільш правильним підходом є випадковий порядок виконання подій, які мають відбудитись в однаковий момент часу. Такий підхід найбільш відповідає тому, що відбувається в реальних системах. Дійсно, між моментами виникнення подій в реальності може бути хоч невелика, але відмінність, яка призводить до того, що відбувається саме одна подія, а не інша. Якщо логіка подій в системі така, що порядок подій чітко визначений, тоді в алгоритмі імітації слід враховувати цей

порядок через встановлений пріоритет виконання подій, момент часу яких співпадає.

Формалізм, який використовується для розробки моделі впливає на розробку алгоритму імітації, насамперед, через набір елементів, з яких складається формальна модель, опис їх стану та правил зміни стану. Отже, наступний важливий крок алгоритму імітації – зміна стану моделі – неможливо розглядати без контексту формалізму, який використаний для опису моделі.

Якщо модель побудована на основі формалізму мережі масового обслуговування, то алгоритм імітації реалізує такі події, як 1) надходження запиту на обслуговування, 2) завершення обробки в каналі обслуговування. Кожна з цих подій в алгоритмі реалізується через зміну стану елементів мережі масового обслуговування таких, як «генератор надходження запитів у $СМО_j$», «черга», «канал обслуговування». Стан моделі визначається станом її елементів (їх перелік див. у таблиці 3.1): для надходження ззовні – це наступний момент надходження ззовні, для черги – це її довжина, для пристрою обслуговування – це момент часу завершення обслуговування, для блокування маршруту – це логічна змінна, що вказує на наявність блокування або його відсутність. Для елементів дуга та розгалуження маршруту не передбачена зміна стану, а для елементу СМО стан визначається станом черги та пристроїв, з яких СМО складається, тому для цього елементу окрема змінна для опису стану не потрібна. У таблиці 3.5 представлено опис змінних та дії, які відтворюють зміну стану. Використані такі позначення:

- t – поточний момент часу;
- ∞ – момент часу, більший за час моделювання;
- g_x – згенероване за заданим (для елементу з індексом x) законом розподілу випадкове число;
- $N_{processed}$ – кількість оброблених на поточний момент часу запитів в мережі масового обслуговування;
- ++ - операція збільшення на 1;
- -- - операція зменшення на 1.

Таблиця 3.5. Зміна стану елементів мережі масового обслуговування.

Елемент	*Змінні, що визначають стан моделі*	*Зміна стану в момент t виникнення події*	
		«надходження запиту на обслуговування у $СМО_j$», $t_{0,j} == t$	*«завершення обробки в r-ому каналі обслуговування $СМО_j$»,* $t_{j,r} == t$
Надходження запитів ззовні	$t_{0,j}$ – наступний момент надходження, N_{input} – кількість надходжень	Якщо маршрут до $СМО_j$ не блокований, то планувати наступний момент надходження $t_{0,j} := t + g_{0,j}$, збільшити кількість надходжень N_{input} ++	Без змін
Пристрій обслуговування	$K_{j,r}$ – стан пристрою, $K_{j,r} \in \{0,1,2\}$ (0, якщо вільний; 1, якщо зайнятий; 2, якщо блокований), $t_{j,r}$ - момент завершення обробки в каналі обслуговування, $t_{j,r} \in \mathbb{R}_+ \cup \{\infty\}$ (якщо не очікуємо завершення обслуговування у наступні моменти часу, то ∞, інакше – невід'ємне ціле число)	Без змін	Якщо маршрут на виході з пристрою не блокований, то звільнити канал обслуговування $K_{j,r} := 0$, $t_{j,r} := \infty$, виконати надходження запиту у наступну по маршруту слідування СМО. Якщо вихід відбується на зовні, то збільшити кількість оброблених запитів в мережі масового обслговування $N_{processed}$ ++.

Елемент	*Змінні, що визначають стан моделі*	*Зміна стану в момент t виникнення події*	
		«надходження запиту на обслуговування у $СМО_j$», $t_{0,j} == t$	*«завершення обробки в r-ому каналі обслуговування $СМО_j$»,* $t_{j,r} == t$
Черга	L – кількість запитів в очікуванні обслуговування, $N_{failure}$ – кількість відмов	Якщо усі канали обслуговування зайняті і є вільні місця в черзі, то L++, інакше $N_{failure}$ ++	Зміна стану черги відбувається після зміни стану пристроя, до якого вона відноситься.Якщо L > 0, то L --, зайняти канал обслуговування $K_{j,r} := 1$, $t_{j,r} := t + g_{j,r}$ (записати в календар подій наступний момент виходу з каналу СМО).

Елемент	*Змінні, що визначають стан моделі*	*Зміна стану в момент t виникнення події*	
		«надходження запиту на обслуговування у $СМО_j$», $t_{0,j} == t$	*«завершення обробки в r-ому каналі обслуговування $СМО_j$»,* $t_{j,r} == t$
Блокування маршруту	b - поточний стан, $b \in B$ (1 – блокований, 0 – не блокований)	Якщо $b == 1$, то збільшити кількість невдалих спроб надходження через блокування маршруту. Оновити поточний стан блокування маршруту $b := B(S,t)$. Якщо $b == 1$, то встановити для елемента, звідки виходить маршрут, стан блокування. Якщо це пристрій обслуговування, то $K_{v,r} == 2$, $t_{v,r} := \infty$. Якщо це надходження ззовні, то $t_0 := \infty$.	Якщо $b == 1$, то оновити поточний стан блокування маршруту $b := B(S,t)$. Якщо $b == 0$ та зняти блокування з елемента, звідки виходить маршрут. Якщо цей елемент - пристрій обслуговування, то запланувати для нього завершення обробки в поточний момент часу $K_{v,q} := 1$, $t_{v,q} := t$. Якщо цей елемент - надходження ззовні, то $t_{0,q} := t$. Оновити поточний стан блокування маршруту $b := B(S,t)$.

У псевдокоді зміна стану мережі масового обслуговування має такий вигляд (використані позначення для стану елементів мережі, які введені вище):

Algorithm 3.2 State transformation E(S,t) of queueing network model

input state of the model S, current time t
foreach j
  if $t_{j,r}$==t then
    if the route from the $device_{j,r}$ is blocked then

```
                    N_blocked++
                    return
                else
                    K_j,r := 0,  t_j,r := ∞,
                    if L_j>0 then
                        L_j --
                        K_j,r := 1, t_j,r := t + g_j,r
                    endif
                    if the route to the device_j,r is blocked then
                        unblock the element from which the route is,
                        plan the event for it in the moment t
                    endif
                endif
            endif
        endforeach
        foreach j
            if t_0,j==t then
                if  route from incoming_0,j is blocked
                    N_blocked++
                    return
                else
                    t_0,j := t + g_0,j
                    N_input ++
                    if L_j==L_max then
                        N_failure ++
                    else
                        if there is no free device then
                            L_j++
                        else
                            foreach device K_j,r
                                if K_j,r == 0 then
                                    K_j,r := 1, t_j,r := t + g_j,r
                                    break
                            endforeach
                        endif
        endforeach
        foreach blocking
```

update blocking condition B(S,t)
endforeach
output next state of model S

Зауважимо, що у вищенаведеному алгоритмі враховано вищий пріоритет події «завершення обробки в j-ому пристрої» у порівнянні з подією «надходження запиту в $СМО_j$». Якщо в циклі foreach індекс j переглядається у зростаючому порядку, то подія з більшим значенням індексу має менший прорітет запуску. Якщо в циклі foreach використовувати випадковий порядок індексу j, то пріоритет запуску подій буде однаковим. Такий цикл у псевдокоді можна описати інструкцією foreach randomly.

В алгоритмі 3.2 умова блокування сформульована для найбільш універсального випадку, коли блокування залежить від поточного стану усіх елементів мережі масового обслуговування. В окремому випадку конкретної задачі умова може виглядати простіше і відповідно реалізована в алгоритмі імітації простіше.

Якщо модель побудована на основі формалізму стохастичної мережі Петрі, то алгоритм імітації реалізує послідовність подій як послідовність запусків переходів. Стан переходу визначається множиною моментів виходу маркерів з переходу. Отже, момент найближчої події визначається за найменшим з усіх моментів виходу маркерів з переходів. При просування часу в момент найближчої події відбувається зміна стану мережі Петрі, а саме, вихід маркерів з переходу, для якого один з його моментів виходу співпадає з поточним моментом часу, та вхід маркерів в переходи, для яких виконана умова запуску переходу. Зауважимо, що в кожний момент часу відбувається саме вихід і вхід маркрів в переходу, на відміну від класичної мережі Петрі, в алгоритмі імітації якої на кожному кроці виконують вхід та вихід маркерів. У класичній мережі Петрі відсутні часові затримки в переході, тому існує така суттєва відмінність в імітації класичної мережі Петрі та мережі Петрі з часовими затримками. Дійсно, для переходу з часовою затримкою між входом та виходом маркерів потрібно відтворювати часову затримку.

Мережі Петрі з багатоканальними переходами мають низку переваг, у тому числі, меншу кількість елементів, які необхідні для представлення, меншу складність алгоритму імітації та компактність графічного представлення. Тому у завданнях, де є для цього підстави, варто використовувати саме алгоритм імітації мережі Петрі з багатоканальними переходами. Наприклад, кілька однакових пристроїв, що обробляють запити зі спільної черги, або захоплення ресурсу, який

наявний у кількості більшій ніж необхідно для обробки одного запиту, представляються багатоканальним переходом простіше, ніж звичайним. А саме, замість багатьох переходів для кожного каналу обробки, в моделі потрібно створити тільки один перехід, який відтворить обробку усіх каналів.

Стан моделі, формалізований опис якої виконано мережею Петрі, у будь-який момент часу визначається станом її позицій та станом її переходів. У таблиці 3.6 наведено зміну стану звичайної мережі Петрі, без урахування багатоканальних переходів, щоб полегшити розуміння зміни стану у випадку багатоканальних переходів. У таблиці 3.7 наведено опис змінних та дії, які відтворюють зміну стану стохастичної мережі Петрі з часовими затримками, з багатональними та конфліктними переходами. Використані такі позначення:

- t – поточний момент часу;
- ∞ – момент часу, більший за час моделювання;
- g_x – згенероване за заданим (для елементу з індексом x) законом розподілу випадкове число;
- M – поточний стан позицій (кількість маркерів в кожній з них);
- E – поточний стан переходів (моменти виходів з переходів);
- $X(M,T)$ – предикат, який визначає виконання умови запуску переходу T для поточного стану позицій M.

Таблиця 3.6. Зміна стану елементів стохастичної мережі Петрі без урахування багатоканальних переходів.

Назва елементу	*Змінні, що визначають стан*	*Зміна стану в момент t*	
		при виході маркерів з переходу $T \mid t_T == t$	*при вході маркерів в перехід* $T \mid X(M,T) == 1$
Позиція P	M_P – кількість маркерів в позиції	Якщо позиція є вихідною для переходу T, то $M_P := M_P + k$ (збільшити кількість маркерів на значення кратності дуги, що з'єднує позицію з переходом).	Якщо позиція є вхідною для переходу, то $M_P := M_P - k$ (зменшити кількість маркерів на значення кратності дуги, що з'єднує позицію з переходом).
Перехід T	E_T - наступний момент виходу маркерів з переходу	$E_T := \infty$ (не очікується вихід з переходу у наступні моменти часу)	$E_T := t + g_T$ (заплановано вихід маркерів з переходу у момент t_T)

Таблиця 3.7. Зміна стану елементів стохастичної мережі Петрі з багатоканальними переходами.

Назва елементу	*Змінні, що визначають стан*	*Зміна стану в момент t*	
		при багатократному виході маркерів з переходу $T \mid t_{T,r} == t$	*при багатократному вході маркерів в перехід* $T \mid X(M,T) == 1$
Позиція P	M_P – кількість маркерів в позиції	Якщо позиція є вихідною для переходу T_j, то $M_P := M_P + k \cdot n$, де n – кількість каналів переходу, для яких $t_{T,r} = = t$ (збільшити кількість маркерів на значення кратності дуги, що з'єднує позицію з переходом, для кожного каналу переходу, для якого момент виходу співпадає з поточним моментом часу).	Якщо позиція є вхідною для переходу, то $M_P := M_P - k \cdot n$, де n – кількість входів до досягнення стану $X(M,T) == 0$ (зменшити кількість маркерів на значення кратності дуги, що з'єднує позицію з переходом, для кожного входу до досягнення стану, в якому для переходу не виконана умова запуску). Зазначимо, що $X(M,T)$ враховує маркірування усіх позицій мережі, тому кількість входів в перехід обумовлюється станом усіх позицій мережі Петрі.
Перехід T	$E_T = \{t_{T,j} \mid j = 1 \dots r\}$ - наступні моменти виходу маркерів з переходу	Якщо $\lvert E_T \rvert > 1$, то $E_T := E_T \backslash \{t_{T,j} \mid t_{T,j} = t\}$, (вилучити з множини моментів виходу усі, для яких момент виходу співпадає з поточним моментом часу). Інакше $E_T := \{\infty\}$ (вказати, що вихід з переходу не планується у наступні моменти часу).	Якщо $\lvert E_T \rvert > 1$, то $E_T := E_T \cup \bigcup_{i=1}^{n} \{t + g_T\}$, де n – кількість входів в перехід до досягнення стану $X(M,T) == 0$ (запланувати виходи з каналів переходу). Інакше $E_T := \bigcup_{i=1}^{n} \{t + g_T\}$ (замінити нескінченність запланованими виходами з каналів.

У псевдокоді зміна стану стохастичної мережі Петрі з багатоканальними переходами має такий вигляд (використані позначення для стану елементів мережі, які введені вище):

Algorithm 3.3 State transformation E(S,t) of stochastic Petri net

```
input state of the model S = (M, T), current time t
foreach randomly transition T
    foreach randomly r
        if t_{T,r}==t then
            foreach place P in list of output places of transition T
                M_P := M_P + k
                if the size of the list E_T is one then
                    t_{T,0} := ∞
                else
                    remove t_{T,r} from the list of E_T
                endif
            endforeach
        endif
    endforeach
endforeach
while ⋁_T X(M, T) == true
    foreach randomly transition T
        if X(M,T) == true then
            foreach place P in list of input places of transition T
                M_P := M_P + k
            if the size of the list E_T is one then
                t_{T,0} := t+g_T
            else
                add t+g_T to the list of E_T
            endif
        endif
    endforeach
endwhile
output next state of the model S  := (M,T)
```

Зауважимо, що в описі псевдокоду зміни стану мережі Петрі використано цикл foreach randomly, тобто для кожного, але у випадковому порядку. При цьому з рівною ймовірністю той чи інший перехід з виконаною умовою виходу

маркерів може опинитись наступним у порядку їх перегляду. У практичній реалізації може бути достатньо виконати операцію перемішування для списку (якщо вона не збільшує занадто складність обчислень).

Якщо багатоканальні переходи не мають спільних позицій, то вони не конфліктують при захопленні маркерів зі своїх вхідних позицій. Тоді фрагмент з входом маркерів у псевдокоді Алгоритму 3.3 можна спростити таким чином:

foreach randomly transition T
  while X(M,T)==true
    foreach place P in list of input places of transition T
      M_P **:=** M_P + k
    if the size of the list E_T is one then
      $t_{T,0}$:= t+g_T
    else
      add t+g_T to the list of E_T
    endif
  endwhile
endforeach

Якщо використовувати звичайний foreach, то порядок перегляду в ньому визначатиме пріоритет запуску переходів: перехід, який на початку списку, матиме вищий пріоритет запуску. Є моделі, для яких такий спосіб запуску переходів може бути корисним.

Якщо модель побудована на основі формалізму Петрі-об'єктної моделі, то її стан повністю визначається станом позицій та переходів, з яких складаються Петрі-об'єкти. Тому зміна стану Петрі-об'єктної моделі виконується зміною стану мереж Петрі усіх її Петрі-об'єктів. Оскільки вихідна позиція одного Петрі-об'єкта може бути вхідною позицією іншого Петрі-об'єкта, то запуск переходів в кожний момент виникнення події виконується до досягнення стану, коли для усіх Петрі-об'єкти досягнуто стан, що для жодного з її переходів не виконана умова запуску. Введемо позначення:

- $\{obj|\ obj = 1, \dots N\}$ – множина ідентифікаторів Петрі-об'єктів, з яких складається модель;
- $S_{obj} = (M_{obj}, T_{obj})$ – стан мережі Петрі об'єкта obj;
- $X_{obj}(S_{obj}) = \bigvee_{T_{obj}} X(M_{obj}, T_{obj})$ – предикат, що визначає наявність переходів з виконаною умовою запуску для заданого стану мережі Петрі S_{obj} об'єкта obj;

– $E_{out}(S_{obj},t)$ – частина перетворення стану мережі Петрі об'єкта obj в момент t, що відповідає виходу маркерів з переходів (цикл обчислень foreach randomly transition T в алгоритмі 3.3);
– $E_{in}(S_{obj},t)$ – частина перетворення стану мережі Петрі об'єкта obj в момент t, що відповідає входу маркерів в переходи (цикл обчислень while в алгоритмі 3.3).

Тоді псевдокод алгоритму імітації Петрі-об'єктної моделі має такий вигляд:

Algorithm 3.4 State transformation E(S,t) of Petri-object model

input state of the model S ={ S_{obj}}, current time t
foreach randomly Petri-object obj
    if t_{obj}==t then
        call output state transformation $E_{out}(S_{obj},t)$
    endif
endforeach
while $\bigvee_{obj}(X_{obj}(S_{obj})) == true$
    foreach randomly obj
        if $X_{obj}(S_{obj})$ == true then
            call input state transformation $E_{in}(S_{obj},t)$
        endif
    endforeach
endwhile
output next state of the model S

Алгоритм імітації Петрі-об'єктної моделі не тільки спрощує та пришвидшує конструювання моделі з окремих фрагментів мереж Петрі, його швидкодія є суттєво вищою за рахунок того, що, по-перше, на кожному кроці імітації пошук найближчого моменту часу відбувається переглядом стану об'єктів замість стану усіх переходів моделі і, по-друге, зміна стану відбувається в межах одного Петрі-об'єкта, що також скорочує обсяг перегляду стану елементів моделі. Дійсно, пошук переходу, для якого виконана умова запуску переходу, відбувається за переглядом тільки переходів, що належать одному Петрі-об'єкту. Якщо предикат для певного об'єкта вже досяг нульового значення, то переходи цього об'єкта вже не переглядатимуться.

Отже, при розробці алгоритму імітації потрібно ретельно враховувати особливості послідовності подій, щоб відтворити їх якнайближче до реальної системи і забезпечити при цьому якнайменшу складність обчислень.

Завершується розробка алгоритму імітації доведенням коректності розробленого алгоритму, що виконується проведенням верифікації алгоритму. Верифікацію алгоритму виконують варіюванням вхідних змінних моделі.

Обов'язково в додатках наводиться фрагмент протоколу подій, який доводить правильність розробленого алгоритму, та таблицю верифікації моделі.

Програмна реалізація імітаційної моделі може бути виконана будь-якою універсальною мовою програмування. При цьому реалізація алгоритму імітації має бути власною. Використання спеціалізованих бібліотек імітації, таких як SimPy або CPNTools, наприклад, не розглядається в межах виконання курсової роботи. Для розробки алгоритму імітації з формалізмом стохастичної мережі Петрі (з часовими затримками, з багатоканальними та конфліктними переходами) можна використовувати бібліотеку класів PetriObjectModelPaint [11]. При цьому в реалізації алгоритму імітації мають бути виконані спрощення, які відповідають умові задачі, з метою зільшення швидкодії алгоритму. Наприклад, якщо формалізований опис виконаний без використання багатоканальних переходів, то відповідну частину коду в алгоритмі імітації слід спростити з урахуванням тільки одноканальних переходів.

Бібліотека Петрі-об'єктного моделювання PetriObjectModelPaint є програмною реалізацією мовою Java технології Петрі-об'єктного моделювання [7]. Для розробки моделі програмне забезпечення такі функції автоматизованого створення моделі:

- створення мережі Петрі засобами графічного редактора;
- анімація запуску переходів на заданому інтервалі часу з виведенням результатім моделювання та протоколу подій;
- запуск моделі на заданому інтервалі часу з виведенням результатім моделювання та протоколу подій;
- збереження мережі Петрі у вигляді Java-методу;
- створення Петрі-об'єктів з заданими параметрами;
- конструювання з них Петрі-об'єктної моделі.

Графічний редактор розробки мережі Петрі оснащений такими інструментами:

- додавання та видалення елементів до зображення,
- редагування параметрів елементів,
- копіювання та вставки елементів,

– виділення групи елементів та перетягування їх,
– виділення групи елементів та видалення їх,
– масштабування зображення моделі,
– перевірка коректності набору елементів у зображенні для створення мережі Петрі.

Вибір засобу реалізації та мови програмування потрібно обгрунтовати, тобто вказати очікувані переваги та недоліки реалізації. Наприклад, мова Python містить потужну біліотеку для реалізації генераторів випадкових чисел з різними законами розподілу, мова C++ дає змогу досягти більшої швидкодії імітації, а мова Java має переваги у використанні багатопочних обчислень.

3.1.4. Проведення експериментального дослідження

При проведенні експериментального дослідження імітаційної моделі використовуємо термінологію, специфічну для математичної статистики та методів планування та проведення експериментів. Факторами називають ті вхідні змінні, вплив яких на вихідну величину досліджується. Вихідна змінна моделі, що досліджується, називається відгуком моделі. Вибір відгуку моделі в курсовій роботі обумовлений метою моделювання, що вказана у завданні. Фактори обираються виконавцем курсової роботи самостійно на основі досвіду, отриманого під час верифікації моделі, результатів моделювання, отриманих при початкових значеннях вхідних змінних та параметрів, а також логічних міркувань, що виходять з розуміння процесу, який відтворюється алгоритмом імітації.

Обов’язковим етапом при проведенні експериментів є дослідження умов проведення одного експерименту, а саме:

– тривалості перехідного періоду, що забезпечує високу точність оцінювання вихідної характеристики;
– кількості повторюваних замірів вихідної змінної моделі, що забезпечують задану точність результату.

Перехідним вважають період, протягом якого модель приходить до своїх сталих значень (рис. 3.1). Значення перехідного періоду, якщо їх не відкинути, сильно впливають на результат оцінювання середнього значення відгуку моделі. Наприклад, якщо стале значення середньої черги в моделі близько 20, а у початковий момент часу значення черги 0, то на початку імітації протягом певного інтервалу часу, середнє значення черги буде поступово збільшуватись. Значення, які спостерігаються на початку імітації, будуть у цьому випадку

зменшувати спостережуване середнє значення відгуку моделі. Тому дослідження перехідного періоду є важливим для досягнення точного результату.

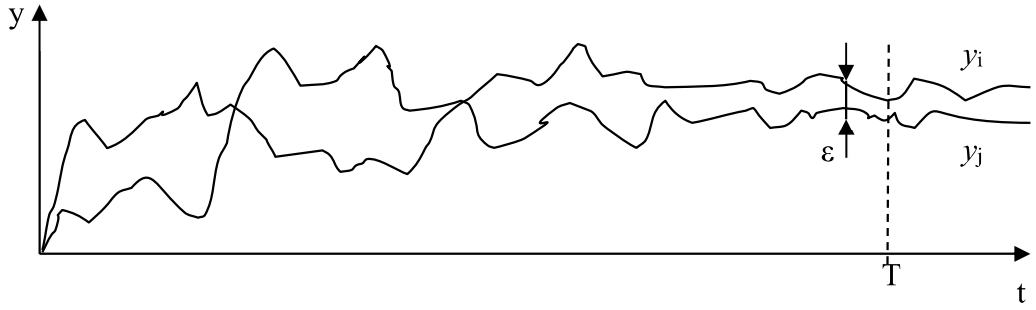


Рисунок 3.1. Визначення тривалості перехідного періоду

Тривалість одного експерименту (в одиницях модельного часу) визначається за пробними спостереженнями динаміки змінювання відгуку моделі з плином часу. Необхідно виконати кілька таких спостережень (щонайменше 4) і визначити момент часу T, після якого різниця у середніх значеннях, обрахованих на цей момент часу, не перевищує задану точність визначення результату:

$$T: \forall i,j \ \left|\overline{y_i(T)} - \overline{y_j(T)}\right| < \varepsilon, \tag{3.1}$$

де ε - задана точність оцінювання, $\overline{y_i(t)}$ - середнє значення відгуку моделі, розраховане за результатом імітації на інтервалі часу [0; t].

Середнє значення відгуку моделі за результатом спостереження моделі на інтервалі часу [0; t], з урахуванням того, що час просувається в алгоритмі на нерівні інтервали часу, потрібно обчислювати за формулою:

$$\overline{y_i(t)} = \frac{1}{n}\frac{\sum_{s=1}^{n} y_{is}\cdot\Delta t_s}{\sum_{s=1}^{n}\Delta t_s}, \ t = \sum_{s=1}^{n}\Delta t_s, \tag{3.2}$$

де $\Delta t_s, s = 1, \dots n$ – інтервали, на які просувається час в алгоритмі імітації, y_{is} – спостережуване значення величини в інтервалі $[t_s; t_{s+1})$.

Стандартне відхилення теж потрібно обчислювати з урахуванням того, що час просувається в алгоритмі на нерівні інтервали часу:

$$\sigma^2(t) = \frac{1}{n-1}\frac{\sum_{s=1}^{n}(y_s-\overline{y})^2\cdot\Delta t_s}{\sum_{s=1}^{n}\Delta t_s}, \ t = \sum_{s=1}^{n}\Delta t_s, \tag{3.3}$$

При визначенні моменту тривалості перехідного періоду можна помилитись у більшу сторону, проте помилка у меншу сторону буде призводити до неточного результату. Якщо умова (3.1) не досягається, то, можливо, сталий стан системи не досягається. Тоді потрібно перейти в експериментальному дослідженні до іншого відгуку моделі або змінити числові характеристики моделі так, щоб сталий режим досягався. Наприклад, при інтенсивності надходження в мережу масового обслуговування більшій за інтенсивність обслуговування, очевидно, буде спостерігатись накопичення в чергах очікування

до нескінченності. Обчислене значення середньої довжини черги буде залежати від часу моделювання, тому такий відгук моделі не є бажаним для дослідження. Навпаки, продуктивність мережі масового обслуговування у таких умовах з часом буде досягати свого максимально можливого значення, яке буде сталим.

Зменшити перехідний період можна встановленням початкового стану моделі, який близький до її сталого стану. Наприклад, для вищезгаданого прикладу починати імітацію слід зі стану черги, коли в ній вже на нульовий момент часу очікують 20 об'єктів.

Зменшити вплив даних перехідного періоду можна також за рахунок збільшення часу моделювання у кілька десятків разів у порівнянні з тривалістю перехідного періоду. Проте такий спосіб витратний, оскільки призводить до надмірних витрат часу на проведення експериментів.

Після того, як тривалість перехідного періоду визначена, можна позбутись впливу на точність даних, зібраних протягом цього періоду одним з трьох згаданих вище способів. На практиці, найбільш дієвими виявляються перші два: встановлення початкового стану якнайближче до того, який спостерігається у сталому режимі, та починати збір даних для статистичної обробки за межами перехідного періоду. Тільки після успішного визначення тривалості перехідного періоду, а значить і необхідного часу моделювання, можна продовжити експериментальне дослідження.

Кількість замірів визначається за формулою, що є наслідком нерівності Чебишева:

$$p = \frac{\sigma^2}{\varepsilon^2(1-\beta)}, \tag{3.4}$$

де σ – стандартне відхилення спостережуваної величини, ε - задана точність оцінювання, β – задана довірча ймовірність оцінювання.

Для цілей імітації, як правило, використовують значення довірчої ймовірності не менше 0,9. У більшості проєктів приймають значення $\beta = 0{,}95$. Тоді формула (3.4) набуває вигляду:

$$p = 20\left(\frac{\sigma}{\varepsilon}\right)^2. \tag{3.5}$$

Звідки слідує, що за вимоги точності $\varepsilon = \sigma$ достатньо 20 повторювань замірів вихідної величини для оцінювання її з точністю σ і довірчою ймовірністю 0,95. За вимоги $\varepsilon = \sigma/2$ кількість необхідних замірів збільшується до 80, а за вимоги $\varepsilon = \sigma/4$ – до 320.

Серія експериментів, яка проводиться, може відрізнятись в залежності від завдання. Результат одного експерименту в серії завжди має бути отриманий як середнє значення повторних замірів досліджуваної вихідної змінної моделі при

фіксованих значеннях вхідних змінних, що забезпечує задану точність оцінювання результату одного експерименту.

Вибір методу дослідження моделі має відповідати меті моделювання, яка вказана у завданні. Якщо у завданні вказано дослідити найліпший варіант чи відшукати параметри, які забезпечують найменше чи найбільше значення вихідної характеристики, то має бути виконаний пошук оптимальних умов або перебором усіх можливих варіантів, або методом градієнтного спуску, або еволюційним методом. Перебір усіх можливих варіантів можна виконати з відкиданням заздалегідь гірших варіантів, щоб скоротити обсяг перебору. Але відкидання певної кількості гірших потрібно обґрунтовувати.

Якщо у завданні вказано, що метою є визначення статистичних характеристик певної вихідної величини, то, окрім визначення середнього статистичного значення та середнього квадратичного відхилення, має бути виконано ідентифікацію закону розподілу випадкової величини. Процес проведення ідентифікації закону розподілу включає такі дії [15]:

– формування масиву спостережуваних значень випадкової величини;
– побудова гістограми частот;
– формування гіпотези про вид закону розподілу;
– визначення параметрів закону розподілу
– перевірка відповідності за критерієм згоди χ^2.

Масив спостережуваних значень формується за результатами запуску імітаційної моделі, при цьому одне значення має бути отримано

Якщо у завданні є дослідження певних модифікацій системи, то потрібно дослідити вплив модифікації на вихідні характеристики системи. Дисперсійний аналіз впливу факторів спрямований на те, щоб надати відповідь чи різниця у значеннях вихідних характеристик спричинена зміною фактору чи вона обумовлена випадковістю. Якщо за фактор прийняти ту чи іншу модифікацію моделі, то за результатом дисперсійного аналізу можна зробити висновок про вплив переходу від однієї модифікації моделі до іншої.

Якщо у завданні тільки вказано дослідження певних вихідних характеристик, то необхідним є експериментальне дослідження впливу вхідних змінних на вихідні змінні моделі. Для такого дослідження можна скористатись або регресійним або дисперсійним аналізом в залежності від того чи змінюється величина, вплив якої досліджується, неперервно чи дискретно.

Якщо з тексту завдання не випливає мета моделювання, то слід обрати метод для використання імітаційної моделі самостійно і узгодити цей вибір з керівником курсової роботи.

Загалом, можливими є такі варіанти використання імітаційної моделі для формулювання висновків про поведінку системи, що моделюється:

- регресійний аналіз;
- дисперсійний аналіз;
- методи групового урахування аргументів;
- методи пошуку оптимальних значень (градієнтний спуск, комбінаторні методи, еволюційні методи, методи штучного інтелекту);
- машинне навчання.

Для застосування цих методів у курсовій роботі можна використати спеціалізовані бібліотеки математичної статистики, машинного навчання, методі оптимізації.

Надамо стислий опис цих методів, достатній для того, щоб здійснити вибір методу, який найліпше підходить для дослідження моделі. Більш детальний опис зазначених методів міститься у навчальному посібнику [15].

Усі методи експериментального дослідження рекомендують використовувати нормалізовані значення факторів, що підвищує точність результатів та якість їх інтерпретації. Нормалізація може бути виконана відносного середнього чи максимального значення фактора. Важливо при цьому, щоб область варіювання усіх факторів після нормалізації була однаковою. Наприклад, до інтервалу [-1; 1] призводять такі формули перетворення значень факторів:

$$x_i = \frac{X_i - X_0}{\Delta_i},\ X_0 = \frac{X_{max} + X_{min}}{2}, \Delta_i = \frac{X_{max} - X_{min}}{2}, \quad (3.6)$$

де $X_i \in [X_{min};\ X_{max}]$ – початкові значення факторів, X_0 – середина інтервалу варіювання, Δ_i – половина довжини інтервалу, $x_i \in [-1;\ 1]$ – нормалізовані значення факторів.

Формули (3.6) широко використовують в теорії інженерного експерименту для планування та проведення факторних експериментів [16].

Регресійний аналіз має на меті визначити кількісно вплив факторів на відгук моделі і потребує виконання таких кроків:

- формування масиву спостережуваних значень;
- формування гіпотези про вид функціональної залежності;
- оцінка значень параметрів функціональної залежності;
- кореляційно-регресійний аналіз функціональної залежності.

Масив спостережуваних значень у випадку дослідження імітаційної моделі формується за результатами проведення експериментів. Для кожного фактору x_i, i= 1,..k рівномірно в межах його діапазону варіювання обираються значення. Для

усіх можливих комбінацій значень факторів проводять експерименти та визначаються результати експериментів y_j, j=1,.. n. Наприклад, у найпростішому випадку лінійної залежності від факторів експерименти достатньо провести при двох значення для кожного фактору – найменшого та найбільшого з діапазону варіювання. Тоді кількість усіх комбінацій цих значень і кількість експериментів, які необхідно провести, складає $n = 2^k$.

Припущення про математичну залежність відгуку моделі y від факторів x_i, i=1,..k, робиться у вигляді математичної функції, яка обов'язково має бути лінійною по шуканих параметрах b_i:

$$f(x, b) = b_0 + b_1 \cdot f_1(x) + b_2 \cdot f_2(x) + \cdots + b_m \cdot f_m(x), \qquad (3.7)$$

де $x = (x_1, x_2, \ldots x_n)$ – вектор факторів, $f_i(x), i = 1,..m$ – диференційовані функції, лінійні або нелійні, але обов'язкові такі, що не є залежними.

Параметри b_i відшукуються так, щоб якнайближче відповідати значенням відгуку моделі $y_j, j = 1,..n$, отриманих в експериментах при значеннях факторів x_j=(x_{1j} ,x_{2j} ,...x_{kj}):

$$\sum_{j=1}^{n}(f(x_j, b) - y_j)^2 \longrightarrow min. \qquad (3.8)$$

Умова (3.7) відома у математичній статистиці як критерій найменших квадратів. Для пошуку параметрів b_i, $i = 1,..m$, що задовольняють умові (3.7), в обчисленнях на комп'ютері зручно використовувати формулу у такому матричному вигляді:

$$b = (X^T X)^{-1} X^T Y, \qquad (3.9)$$

де $b = \begin{pmatrix} b_0 \\ b_1 \\ \cdots \\ b_m \end{pmatrix}$, $X = \begin{pmatrix} 1 & f_1(x_1) & \cdots & f_m(x_1) \\ 1 & f_1(x_2) & \cdots & f_m(x_2) \\ \cdots & \cdots & \cdots & \cdots \\ 1 & f_1(x_n) & \cdots & f_m(x_n) \end{pmatrix}$, $Y = \begin{pmatrix} y_1 \\ y_2 \\ \cdots \\ y_n \end{pmatrix}$.

Успішність визначення функціональної залежності оцінюють за результатами виконання кореляційно-регресійного аналізу. Розрахований показник індексу кореляції свідчить про близькість суми квадратів відхилень спостережуваних в експериментах значень відгуку моделі і обрахованих за рівнянням регресії від середнього значення відгуку моделі в експериментах. Чим ближче значення індексу кореляції до 1, тим більш успішним є результат пошуку регресійної залежності. При значеннях індексу кореляції менших за 0,7 слід прийняти рішення про неуспішність пошуку функціональної залежності, і або повторити пошук при більшій кількості даних, або перейти до більш складної функціональної залежності.

Дисперсійний аналіз має на меті визначити якісно вплив факторів на відгук моделі. Для його проведення потрібно виконати такі дії:

– формування масиву спостережуваних значень;

– оцінювання впливу за критерієм Фішера.

У випадку дослідження імітаційної моделі масив спостережуваних значень формується за результатами проведення експериментів. Для кожного фактору обирається однакова кількість q значень в його діапазоні варіювання. На відміну від регресійного аналізу, фактори можуть приймати якісні, тобто некількісні, значення, такі як «з управлінням / без управління», «високий/середній/низький», «модифікований / не модифікований» тощо. Для кожного значення проводиться експеримент, результатом якого є достатньо точне значення відгуку моделі. Позначимо $y_{js}, s = 1,..p$ – значення відгуку моделі, отримане, $\overline{y_j}$, $j = 1, ...q$ – значення відгуку моделі, отримане в j-ому експерименті для j-ого значення фактора, $\overline{y}$ – середнє значення всіх $\overline{y_j}$.

Факторна дисперсія розраховується як сума квадратів відхилень значень відгуку моделі, отриманих у кожному j-ому експерименті, від загального середнього значення, поділена на $q \cdot p - 1$:

$$D_f = \frac{p \cdot \sum_{j=1}^{q} (\overline{y_j} - \overline{y})^2}{q \cdot p - 1}. \quad (3.10)$$

Залишкова дисперсія розраховується як сума квадратів відхилень значень відгуку моделі, отриманих для кожного i-ого заміру значення фактора, від середнього значення, розрахованого для відповідного j-ого експерименту, поділена на $q \cdot p - 1$:

$$D_r = \frac{\sum_{j=1}^{q} \sum_{s=1}^{p} (y_{js} - \overline{y_j})^2}{q(p-1)}. \quad (3.11)$$

Близькість факторної і залишкової дисперсій означає, що різниця у значеннях відгуку моделі, отриманих при різних значеннях фактора, обумовлена швидше випадковістю, ніж зміною значення фактора. І, навпаки, відмінність вказаних сум квадратів відхилень свідчить про суттєвий вплив фактора на відгук моделі. Близькість / відмінність дисперсій оцінюється за критерієм Фішера.

Методи групового урахування аргументів (МГУА) призначені для пошуку математичної функції, що достатньо точно описує залежність між факторами та відгуком моделі. На відміну від регресійного аналізу ці методи виходять з того, що неможливо за тими ж даними, що використовувались для пошуку параметрів функціональної залежності, оцінювати якість знайденої моделі. Тому уся множини даних для застосування цих методів поділяється на навчальну та перевірну послідовності. Дані, які віднесені до перевірної послідовності, не використовують для пошуку параметрів моделі, щоб забезпечити справедливе оцінювання моделей. Навчальні дані слугують для визначення параметрів моделей-претендентів. Рекомендовано обирати співвідношення кількості даних

навчальної та перевірної послідовності 2/1, але цей параметр може бути предметом окремого дослідження.

Отже, для кожної моделі з множини моделей претендентів розраховують її параметри. Спосіб обчислення їх аналогічний тому, що використовують при пошуку параметрів регресійної кривої. Тому так само, як і для регресійного аналізу, моделі мають бути лінійно залежними від шуканих параметрів, а дані, які використовуються для пошуку параметрів - нормалізовані. Далі виконується вибір найліпшої моделі за критерієм, що використовує дані перевірної послідовності даних. Модель, для якої досягається мінімум критерія, визнається найліпшою і є результатом обчислень.

У комбінаторному (однорядному) алгоритмі МГУА здійснюється перебір усіх моделей з множини моделей-претендентів, а множина моделей-претендентів формується з усіх можливих комбінацій доданків найскладнішої моделі. Комбінаторний алгоритм може призводити до надто великої кількості моделей в множині моделей-претендентів і відповідно надто довгих обчислень, тому були винайдені алгоритми, які виконують неповний перебір моделей.

У багаторядному алгоритмі МГУА складність моделей збільшують поступово, допоки зменшується досягнуте значення критерію. У наступний ряд проходять тільки N найуспішніших моделей. При цьому дані, розраховані за моделями першого ряду, слугують вхідними даними для моделей другого ряду, і т.д. Модель найліпшого ряду, яка має найменше значення критерію визнається найліпшою.

Перевагою застосування алгоритмів МГУА є об’єктивність вибору функціональної залежності, а недоліком їх – значні обсяги обчислень та необхідність налаштовувати параметри алгоритму, такі як множину моделей-претендентів, поділ даних на частини, критерій вибору найліпшої моделі, послідовність перебору моделей (так, щоб вона відповідала умові зростання складності моделі), для кожної специфічної задачі. Вважається, що досягнення значення критерію 0,05 (на даних перевірної послідовності) свідчить про успішність пошуку моделі.

Методи пошуку оптимальних значень встановлюють значення факторів, за яких досягається мінімум/максимум відгуку моделі. Якщо область варіювання факторів обмежена, то оптимальне значення відгуку завжди існує. Якщо фактори приймають дискретні значення і кількість усіх можливих комбінацій значень факторів невелика, то оптимальне значення відгуку може бути обране перебором усіх можливих варіантів. Якщо ж такий пошук вимагатиме значних витрат часу,

то логічними міркуваннями можна відкинути найбільш неперспективні варіанти і продовжити пошук для меншої кількості варіантів.

Якщо фактор може приймати будь-яке значення з інтервалу, то перебір усіх можливих його значень неможливий і пошук ускладнюється. Для проведення експериментів обирають кінцеву кількість значень в межах інтервалу, а значення визначають так, щоб з рівним кроком покрити весь інтервал варіювання, включно з його границями:

$$x_i = -1 + \frac{2}{n-1} \cdot i, \quad i = 0,..n-1, \tag{3.12}$$

де $x_i \in [-1;\ 1]$ - нормалізовані значення фактору (див. (3.6)), n – кількість значень.

Кількість значень для кожного фактора обирають так, щоб перебір усіх можливих комбінацій значень факторів не призводив до надмірної кількості експериментів. Далі проводять експерименти при кожній комбінації значень факторів та визначають ту з них, за якої досягається оптимальне значення відгуку моделі. Величина $\frac{2}{n-1}$ буде в цьому випадку є оцінкою точності, з якою виконаний пошук оптимальних значень факторів. Нагадаємо, що кількість усіх можливих комбінацій знаходиться добутком усіх можливих значень факторів:

$$\prod_{i=1}^{k} q_i, \tag{3.12}$$

де q_i – кількість усіх можливих значень i-ого фактора.

Наприклад, якщо для одно фактора визначили 10 можливих значень, для іншого – 2, а для третього – 100, то кількість експериментів повного перебору складатиме 20000. Навіть якщо один експеримент триває близько 2 секунди, то витрати часу на проведення серії експериментів (10000 секунд) можуть бути надмірними.

Кількість значень, при яких проводяться експерименти можна суттєво скоротити, якщо використовувати еволюційні методи пошуку, які використовують евристику еволюційного відбору для направленого пошуку оптимальних значень факторів. Одна комбінація факторів, з якою проводять експеримент, в термінах еволюційного методу – це особина популяції, а результат експерименту з цією комбінацією факторів – оцінка життєдіяльності особини. На першому кроці еволюційного методу генерують N особин із значеннями факторів, які випадково згенеровані в області варіювання факторів. Експериментально оцінюють життєдіяльність кожної особини і обрають частину з них, що мають найліпші показники життєдіяльності, для переходу в наступну популяцію. Інша частина має бути отримати за результатом схрещування та

мутації цих найліпших особин. Інколи до них додають також нові особини, випадково згенеровані з метою додати різноманітності.

Алгоритми схрещування і мутації намагаються побудувати так, щоб отримати якомога більш придатну до життя особину. Якщо вдається врахувати специфіку задачі, то успішність методу буде вражаючою: дуже швидко алгоритм знаходитиме оптимальні значення. Наприклад, в задачі про дорожній рух на ділянці, керованій світлофорами, логічно не розривати пари сигналів зеленого та червоного світла, які показують високі показники якості управління. Тоді в алгоритмі схрещування перемішують пари цих значень для різних перехресть ділянки руху, щоб отримати особину зі ще більш високими показниками. Мутацію застосовують до особини, що отримана схрещуванням, додаванням випадкового відхилення, щоб додати різноманітності новій популяції. Величина відхилення має бути невеликою у порівнянні з самими значеннями (до 10 %), інакше мутація буде надто далеко відкидати особини від найліпших і тільки сповільнювати пошук.

У найпростішому випадку схрещування двох особин організовують обміном їх частин:

$$\begin{pmatrix}\{a_i|\ i=1,..k\}\\ \{b_i|\ i=1,..k\}\end{pmatrix} \Longrightarrow \begin{pmatrix}\left\{c_i|\ c_i=a_i, i=1,..\frac{k}{2}, c_i=b_i, i=\frac{k}{2},..n\right\}\\ \left\{d_i|\ d_i=b_i, i=1,..\frac{k}{2}, d_i=a, i=\frac{k}{2},..n\right\}\end{pmatrix}, \quad (3.13)$$

де $\begin{pmatrix}a\\ b\end{pmatrix}$ – пара особин, що схрещуються, $\begin{pmatrix}c\\ d\end{pmatrix}$ – пара особин, утворена в результаті схрещування.

Мутацію, у найпростішому випадку, виконують генеруванням випадково (рівномірно) одного значення з множини {-1; 1; 0}, що визначатиме, відповідно, відхилення віднімається, додається або є нульовим. У випадку нормалізованих факторів $x_i \in [-1;\ 1], i=1,..k$ відхилення можна встановити 0,1, наприклад. Тоді перетворення за алгоритмом мутації має вигляд:

$$\{c_i|\ i=1,..k\} \Longrightarrow \{c_i+0{,}1r_i|\ r_i \in \{-1{,}1{,}0\}, i=1,..k\},$$

деc_i – особина, утворена схрещуванням, r_i є випадково згенерованим числом за рівномірним розподілом з множини $\{-1{,}1{,}0\}$ і згенерованим для кожного фактору c_i.

Успішність еволюційного методу сильно залежить від розміру популяції та визначення алгоритмів схрещування та мутації. Розмір популяції має бути достатньо великим, щоб забезпечити більшу ймовірність потрапляння в оптимальні значення. Проте великий розмір популяції призведе до збільшення кількості експериментів і збільшення витрат часу на його проведення. Отже,

доводиться балансувати між розміром популяції та кількістю поколінь, яке знадобиться для пошуку оптимальних значень.

Іншим варіантом пошуку оптимальних значень може бути побудова нелінійної функціональної залежності відгуку моделі від факторів регресійним аналізом або методами групового урахування аргументів та використання цієї функціональної залежності для пошуку оптимальних значень звичайними методами математичного аналізу.

Методи машинного навчання можуть використовувати дані для свого навчання, отримані за результатами проведення експериментів з імітаційною моделлю. Якщо проведення імітації є довготривалим процесом, то використовувати в управлінні безпосередньо імітацію неможливо. Навчена нейромережа може достатньо точно відтворювати результати, отримані в імітації. І, хоч процес навчання є довготривалим, проте після навчання обчислення результату на нейромережі є швидким. Такий спосіб використання імітаційних моделей набуває все більшого поширення, оскільки спостережувати і збирати дані з імітаційної моделі набагато швидше, ніж з реальної системи, а інколи варіювання факторів і, відповідно, отримання результатів експериментів у реальній системі взагалі є неможливим.

3.1.5. Інтерпретація результатів моделювання

У цьому розділі курсової роботи акумулюється інформація, отримана з моделі в процесі її розробки та дослідження. Якщо були прийняті рішення про зміни в моделі у порівнянні з початковим текстом завдання, то вони вказуються разом з обґрунтуванням причин, які привели до такого рішення.

Узагальнено у текстовому вигляді наводяться результати верифікації моделі, обґрунтовується вибір факторів для проведення експериментального дослідження та наводяться результати експериментального дослідження. Приділяється увага виявленим залежностям та закономірностям між вхідними та вихідними змінними моделі. Усі висновки щодо функціонування системи набувають словесного трактування і мають бути написані так, щоб не обізнана в моделюванні людина, якою найчастіше є замовник моделі, могла зрозуміти отримані результати.

На основі отриманих результатів моделювання формулюються висновки про якість функціонування системи, що моделюється, та надаються пропозиції щодо поліпшення її функціонування. Для пропонованих змін наводяться результати імітації та оцінюється, які з характеристик системи і на скільки поліпшені, а які, навпаки, погіршились. Наприкінці робиться висновок про

доцільність запропонованих змін. Доречним у цьому розділі є також формулювання пропозицій до подальшого дослідження системи.

3.2. Графік виконання курсової роботи

Виконання курсової роботи поділено на частини, які можуть здаватись студентом по мірі виконання курсової роботи: розробка концептуальної моделі, розробка формалізованої моделі, реалізація імітаційної моделі, проведення експериментального дослідження на моделі. Викладач має можливість вже на перших кроках виконання завдання виявити та вказати на помилки в розробці.

Рекомендації щодо термінів кожного етапу виконання курсової роботи наведені у таблиці 3.7. Графік виконання курсової роботи має узгоджуватись з керівником курсової роботи і відповідати запланованій даті захисту.

Таблиця 3.7. Графік виконання курсової роботи

Тиждень навчального семестру, відведений на виконання етапу роботи	*Назва етапу виконання роботи*	*Форма звітності*
1-4	Отримання індивідуального завдання на курсову роботу	Затвердження теми
5	Розробка концептуальної модель системи	Оформлення відповідного розділу курсосї роботи
6-7	Розробка формалізованої моделі системи	Модуль програмного забезпечення
8-10	Алгоритмізація моделі системи та її програмна реалізація	Модуль програмного забезпечення
11-12	Експериментальне дослідження моделі системи	Результати тестування у вигляді тексту чи таблиці
13	Інтерпретація результатів моделювання, формулювання висновків та пропозицій	Результати експериментального дослідження у вигляді тексту чи таблиці
14-15	Оформлення пояснювальної записки	Текст пояснювальної записки, оформлений у відповідності до вимог
16	Захист курсової роботи.	Прилюдний захист з презентацією результатів курсової роботи